In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
data = pd.read_csv('heart.csv')
print(data.head())
```

Out:
```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang   oldpeak  \
0   63    0   3       132   272    1        2      136      1  1.075449
1   76    1   2       132   157    0        1      129      0  0.835734
2   53    1   3        97   166    0        2      166      0  0.638198
3   39    0   1       133   249    0        2      106      0  0.684821
4   67    0   0       118   209    0        1      116      0  1.317656

   slope  ca  thal  target
0      1   3     3       0
1      2   0     2       0
2      0   4     0       0
3      1   0     2       0
4      1   4     2       0
```

In [3]:
```
data.info()
```

Out:
```
<class 'pandas.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
```

```
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

In [4]:

```
print(data.shape)
print(data.columns.tolist())
```

Out:

```
(1025, 14)
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak',
'slope', 'ca', 'thal', 'target']
```

In [5]:

```
print(data.describe())
```

Out:

```
               age          sex          cp     trestbps         chol  \
count  1025.000000  1025.000000  1025.00000  1025.000000  1025.000000
mean     52.377561     0.710244     1.52000   130.250732   238.579512
std      15.897772     0.453871     1.13616    19.783461    49.832843
min      25.000000     0.000000     0.00000    62.000000    73.000000
25%      39.000000     0.000000     1.00000   117.000000   202.000000
50%      52.000000     1.000000     1.00000   131.000000   238.000000
75%      66.000000     1.000000     3.00000   144.000000   275.000000
max      79.000000     1.000000     3.00000   199.000000   386.000000

               fbs      restecg      thalach        exang      oldpeak  \
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000
mean      0.238049     1.022439   138.296585     0.268293     0.953093
std       0.426097     0.803121    30.150402     0.443287     0.915476
min       0.000000     0.000000    53.000000     0.000000     0.000053
25%       0.000000     0.000000   119.000000     0.000000     0.308052
50%       0.000000     1.000000   138.000000     0.000000     0.685615
75%       0.000000     2.000000   158.000000     1.000000     1.315933
max       1.000000     2.000000   232.000000     1.000000     6.583731

             slope           ca         thal       target
count  1025.000000  1025.000000  1025.000000  1025.000000
mean      1.004878     1.999024     1.510244     0.029268
std       0.808268     1.404860     1.119842     0.168640
min       0.000000     0.000000     0.000000     0.000000
```

```
25%       0.000000     1.000000     1.000000     0.000000
50%       1.000000     2.000000     2.000000     0.000000
75%       2.000000     3.000000     3.000000     0.000000
max       2.000000     4.000000     3.000000     1.000000
```

In [6]:
```
print(data.isnull().sum())
```

Out:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

In [7]:
```
X = data.drop('target', axis=1)
y = data['target']
```

In [8]:
```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, rec
all_score, f1_score, roc_auc_score
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st
ate=42)
print(f"Training set: {len(X_train)}")
print(f"Testing set: {len(X_test)}")
print(f"Training target: {len(y_train)}")
print(f"Testing target: {len(y_test)}")
```

Out:

```
Training set: 717
Testing set: 308
Training target: 717
Testing target: 308
```

In [10]:

```
model = GaussianNB()
model.fit(X_train, y_train)
```

Out:

```
▾ GaussianNB
▶ Parameters
```

In [11]:

```
y_pred = model.predict(X_test)
print("Predicted values:")
print(y_pred)
```

Out:

```
Predicted values:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0]
```

In [12]:

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
y_pred_proba = model.predict_proba(X_test)[:, 1]
roc_auc = roc_auc_score(y_test, y_pred_proba)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
```

Out:

```
Accuracy: 0.9708
Precision: 0.0000
Recall: 0.0000
F1-Score: 0.0000
ROC-AUC: 0.8880
```

In [13]:

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

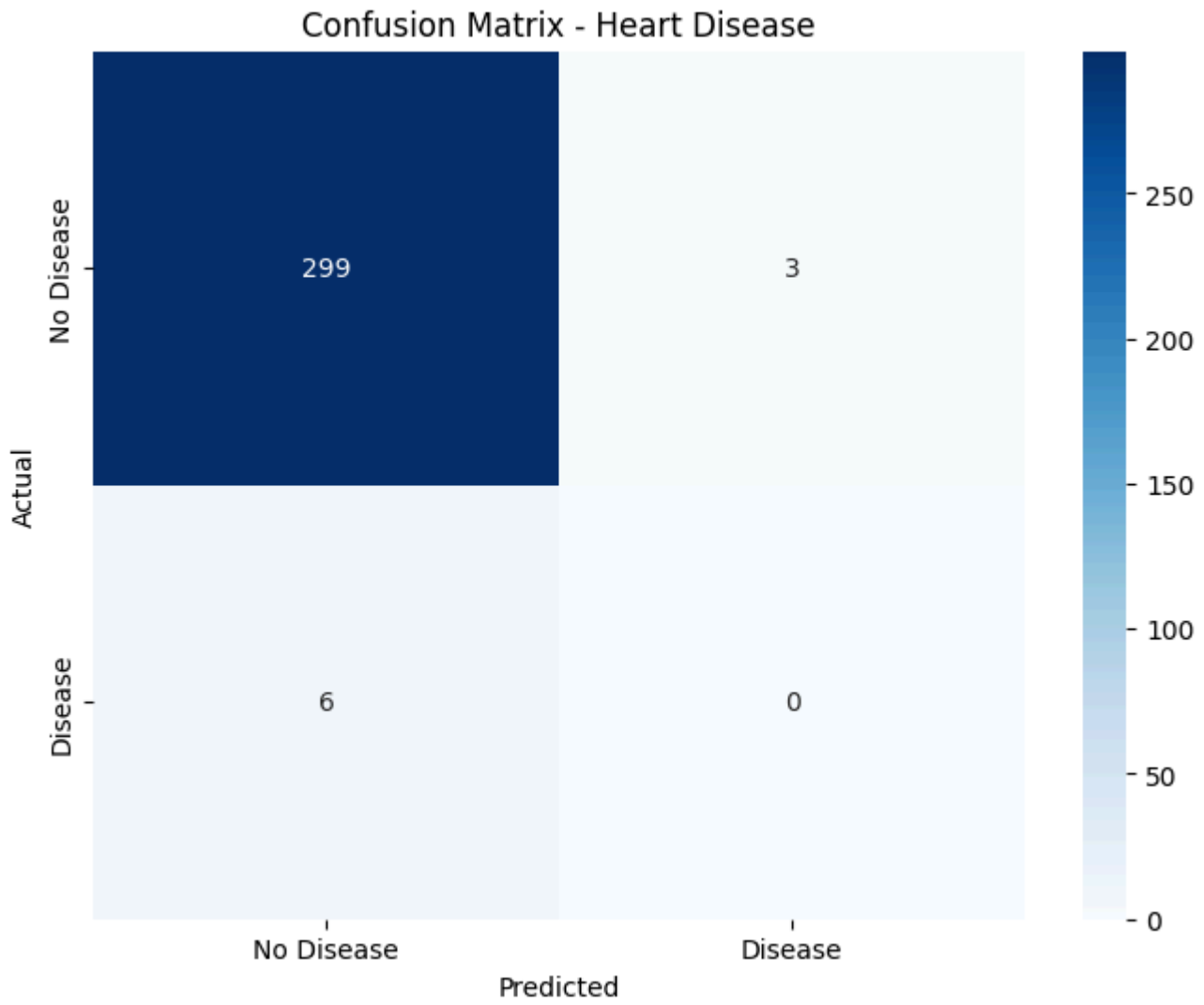
Out:

```
Confusion Matrix:
[[299   3]
 [  6   0]]
```

In [14]:

```
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["No Disease", "Disease"],
            yticklabels=["No Disease", "Disease"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Heart Disease")
plt.show()
```

Out:



In [15]:

```
print(f"True Negatives (TN): {cm[0, 0]}")
print(f"False Positives (FP): {cm[0, 1]}")
print(f"False Negatives (FN): {cm[1, 0]}")
print(f"True Positives (TP): {cm[1, 1]}")

specificity = cm[0, 0] / (cm[0, 0] + cm[0, 1])
sensitivity = cm[1, 1] / (cm[1, 1] + cm[1, 0])
print(f"\nSensitivity (True Positive Rate): {sensitivity:.4f}")
print(f"Specificity (True Negative Rate): {specificity:.4f}")
```

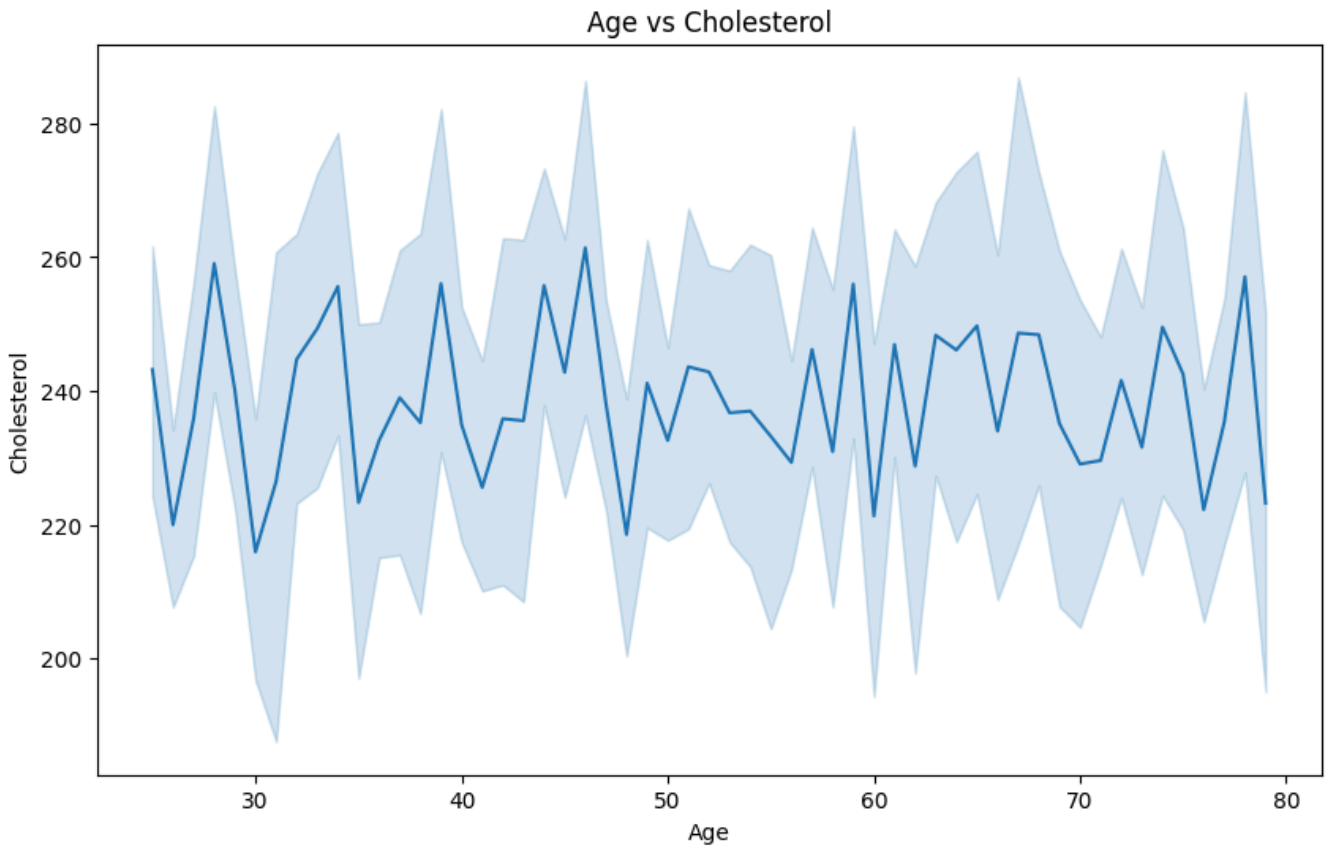
Out:

```
True Negatives (TN): 299
False Positives (FP): 3
False Negatives (FN): 6
True Positives (TP): 0

Sensitivity (True Positive Rate): 0.0000
Specificity (True Negative Rate): 0.9901
```

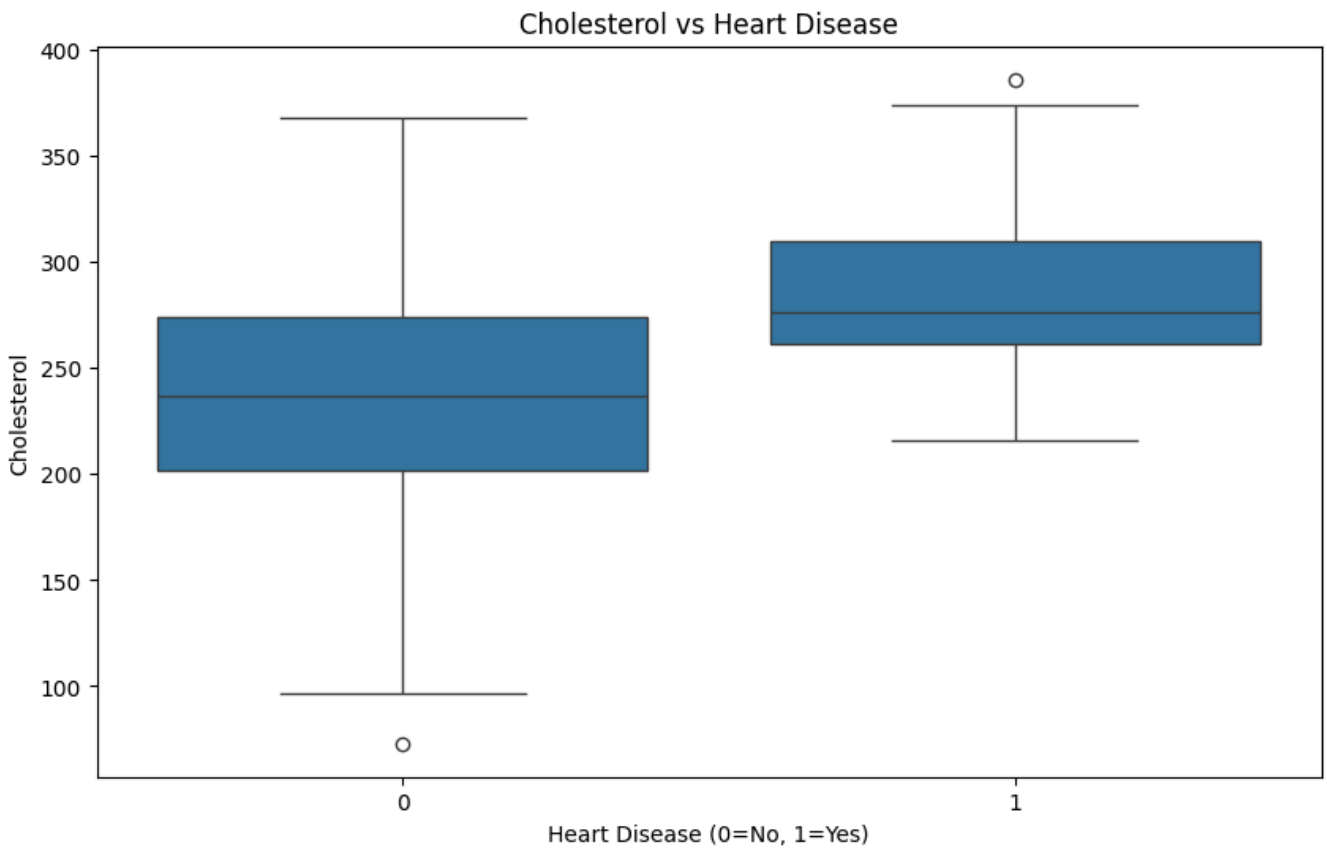
In [16]:
```
plt.figure(figsize=(10, 6))
sns.lineplot(x="age", y="chol", data=data)
plt.title("Age vs Cholesterol")
plt.xlabel("Age")
plt.ylabel("Cholesterol")
plt.show()
```

Out:



In [17]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x="target", y="chol", data=data)
plt.xlabel("Heart Disease (0=No, 1=Yes)")
plt.ylabel("Cholesterol")
plt.title("Cholesterol vs Heart Disease")
plt.show()
```

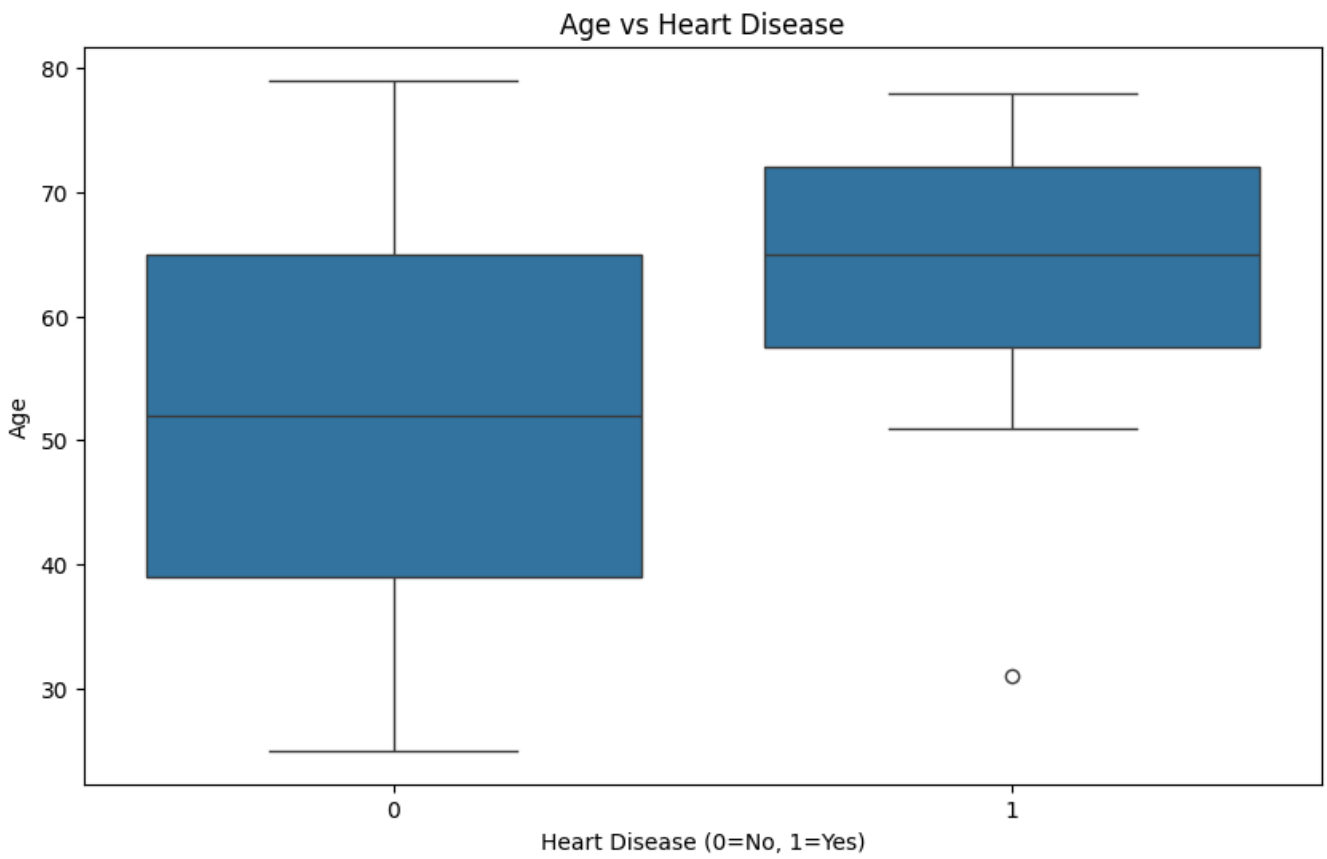
Out:



In [18]:

```
plt.figure(figsize=(10, 6))
sns.boxplot(x="target", y="age", data=data)
plt.xlabel("Heart Disease (0=No, 1=Yes)")
plt.ylabel("Age")
plt.title("Age vs Heart Disease")
plt.show()
```

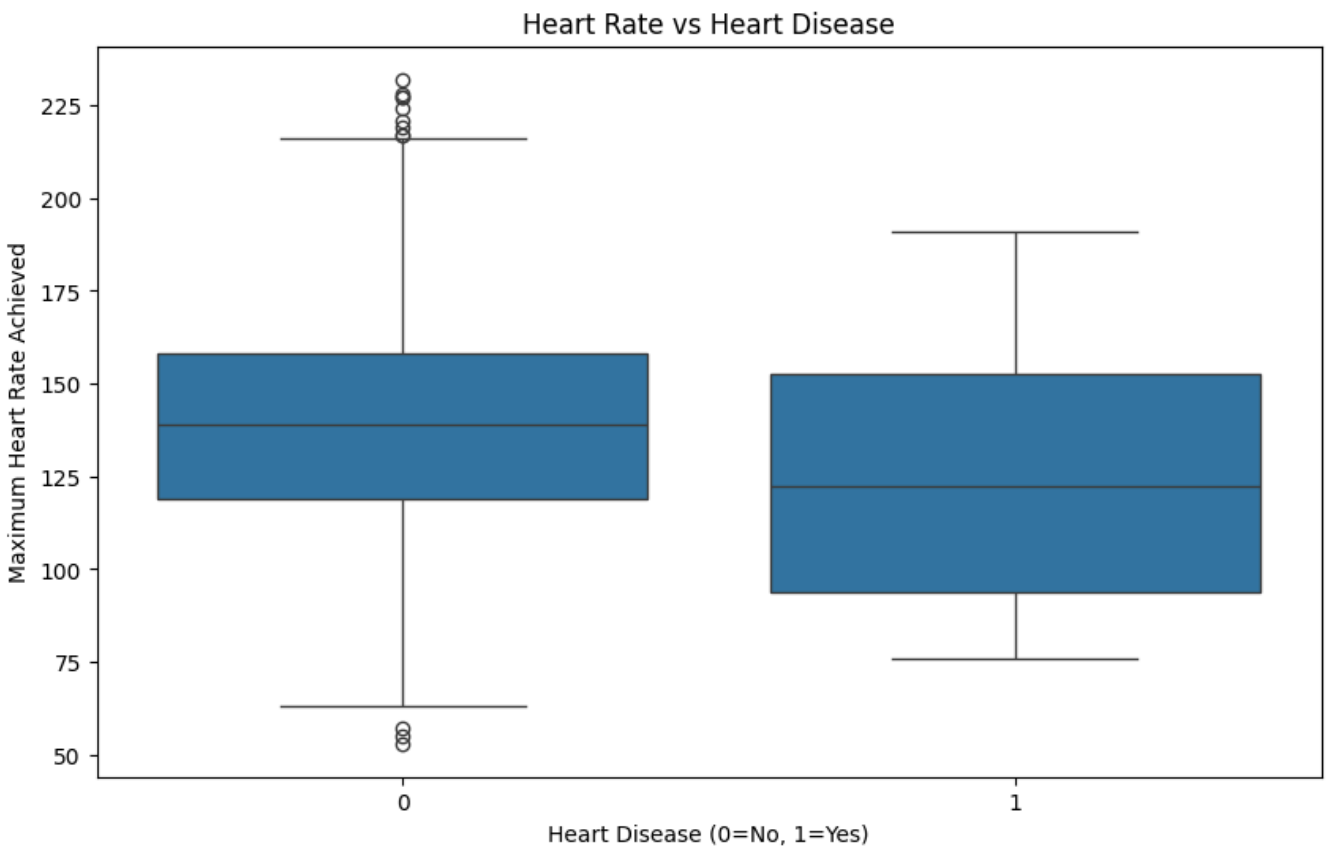
Out:



In [19]:

```
plt.figure(figsize=(10, 6))
sns.boxplot(x="target", y="thalach", data=data)
plt.xlabel("Heart Disease (0=No, 1=Yes)")
plt.ylabel("Maximum Heart Rate Achieved")
plt.title("Heart Rate vs Heart Disease")
plt.show()
```

Out:



In [20]:

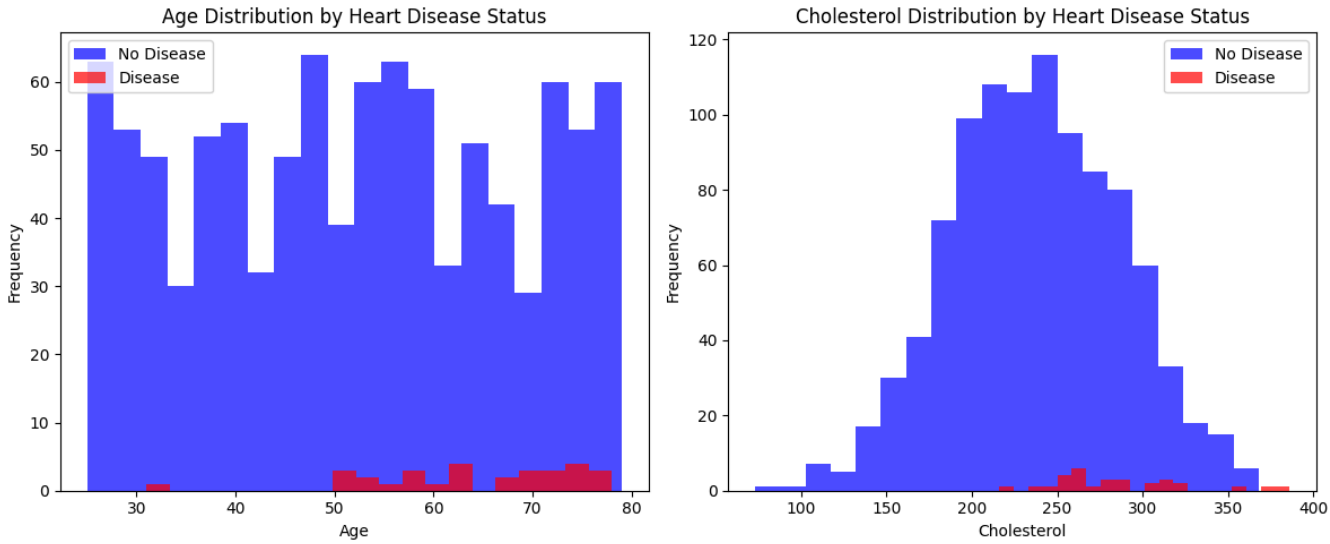
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(data[data['target']==0]['age'], bins=20, alpha=0.7, label='No Disease',
color='blue')
plt.hist(data[data['target']==1]['age'], bins=20, alpha=0.7, label='Disease', col
or='red')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Distribution by Heart Disease Status')
plt.legend()

plt.subplot(1, 2, 2)
plt.hist(data[data['target']==0]['chol'], bins=20, alpha=0.7, label='No Disease',
color='blue')
plt.hist(data[data['target']==1]['chol'], bins=20, alpha=0.7, label='Disease', co
lor='red')
plt.xlabel('Cholesterol')
plt.ylabel('Frequency')
plt.title('Cholesterol Distribution by Heart Disease Status')
plt.legend()

plt.tight_layout()
plt.show()
```

Out:



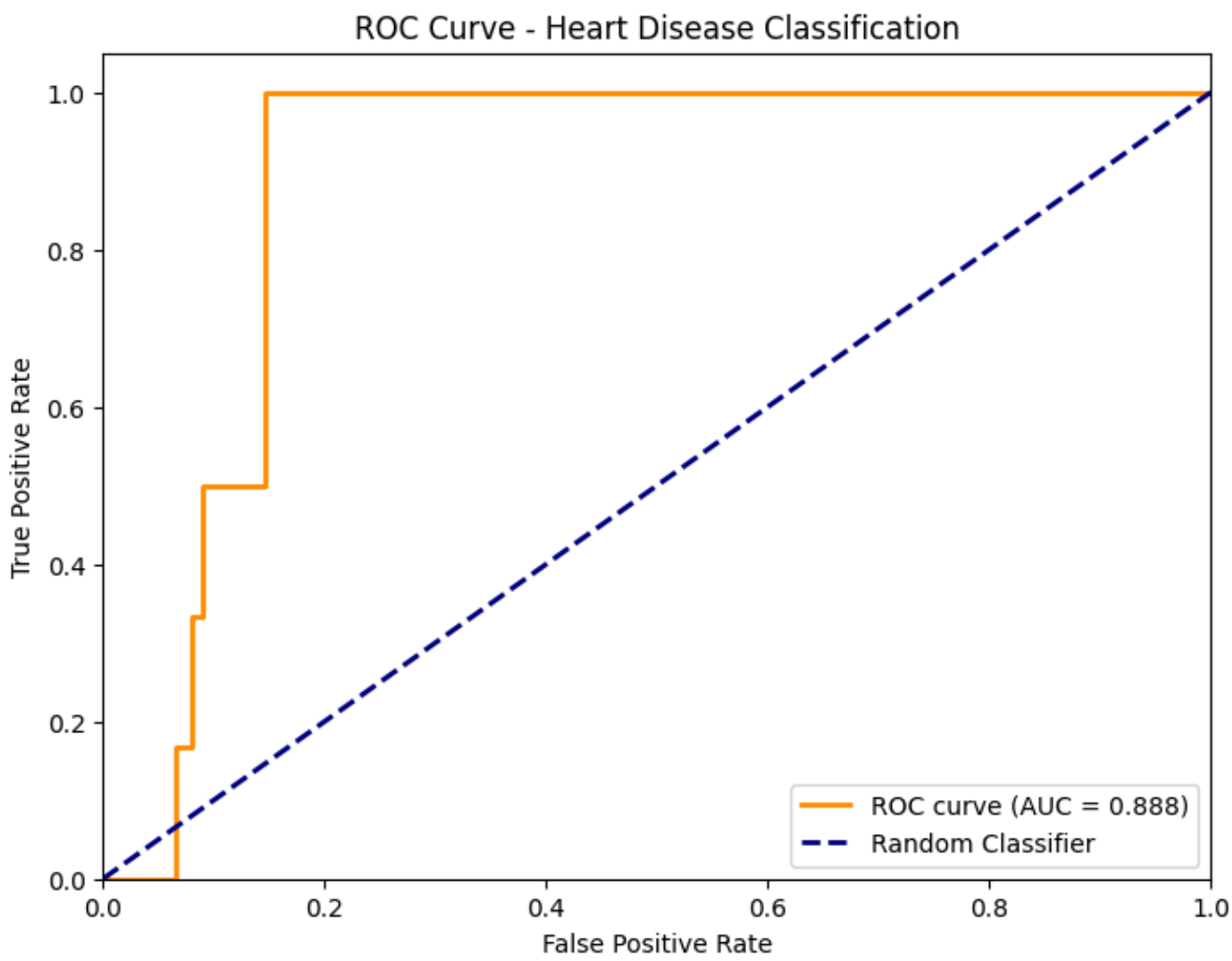
In [21]:

```
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc_value = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc_va
lue:.3f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Classi
fier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Heart Disease Classification')
plt.legend(loc="lower right")
plt.show()
```

Out:



In [22]:

```
y_train_pred = model.predict(X_train)
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_pred)

print(f"Training Accuracy: {train_accuracy:.4f}")
print(f"Testing Accuracy: {test_accuracy:.4f}")
print(f"Difference: {abs(train_accuracy - test_accuracy):.4f}")
```

Out:

```
Training Accuracy: 0.9582
Testing Accuracy: 0.9708
Difference: 0.0126
```

In [23]:

```
metrics = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1,
    'ROC-AUC': roc_auc,
```

```
    'Sensitivity': sensitivity,
    'Specificity': specificity
}

print("\n" + "="*50)
print("MODEL PERFORMANCE SUMMARY")
print("="*50)
for metric, value in metrics.items():
    print(f"{metric}: {value:.4f}")
print("="*50)
```

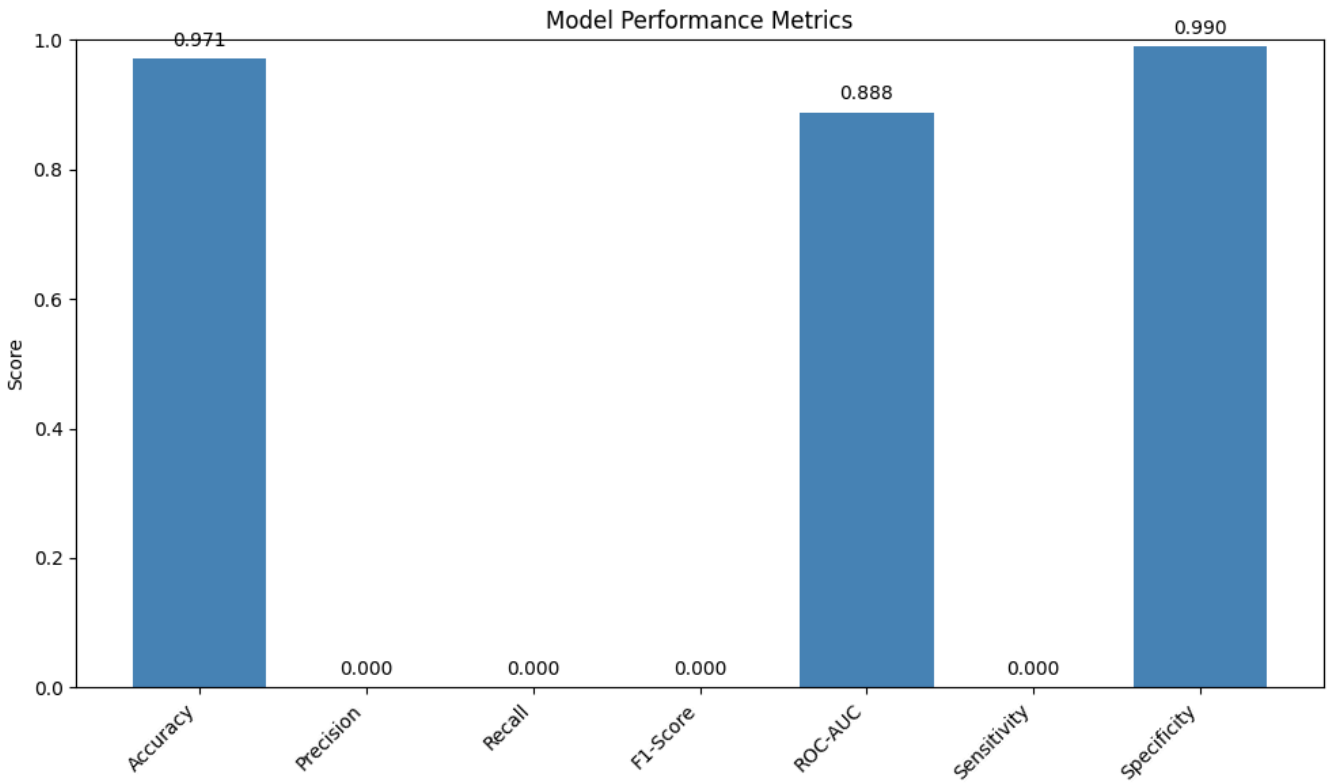
Out:

```
==================================================
MODEL PERFORMANCE SUMMARY
==================================================
Accuracy: 0.9708
Precision: 0.0000
Recall: 0.0000
F1-Score: 0.0000
ROC-AUC: 0.8880
Sensitivity: 0.0000
Specificity: 0.9901
==================================================
```

In [24]:

```
metrics_df = pd.DataFrame(list(metrics.items()), columns=['Metric', 'Value'])
plt.figure(figsize=(10, 6))
plt.bar(metrics_df['Metric'], metrics_df['Value'], color='steelblue')
plt.ylabel('Score')
plt.title('Model Performance Metrics')
plt.ylim([0, 1])
plt.xticks(rotation=45, ha='right')
for i, v in enumerate(metrics_df['Value']):
    plt.text(i, v + 0.02, f'{v:.3f}', ha='center')
plt.tight_layout()
plt.show()
```

Out:



In [25]:

```
sample_data = X_test.iloc[:10].copy()
sample_predictions = model.predict(sample_data)
sample_probabilities = model.predict_proba(sample_data)

results_df = pd.DataFrame({
    'Actual': y_test.iloc[:10].values,
    'Predicted': sample_predictions,
    'Prob_No_Disease': sample_probabilities[:, 0],
    'Prob_Disease': sample_probabilities[:, 1],
    'Correct': sample_predictions == y_test.iloc[:10].values
})

print("\nSample Predictions (First 10 test samples):")
print(results_df)
```

Out:

```
Sample Predictions (First 10 test samples):
   Actual  Predicted  Prob_No_Disease  Prob_Disease  Correct
0       0          0         0.987533      0.012467     True
1       0          0         0.991508      0.008492     True
2       0          0         0.996233      0.003767     True
3       0          0         0.991668      0.008332     True
4       0          0         0.995434      0.004566     True
5       0          0         0.597123      0.402877     True
6       0          0         0.998799      0.001201     True
7       0          0         0.986638      0.013362     True
```

```
8       0          0         0.935342      0.064658     True
9       0          0         0.999960      0.000040     True
```

In [26]:

```python
print(f"\nTotal Test Samples: {len(y_test)}")
print(f"Correct Predictions: {np.sum(y_pred == y_test.values)}")
print(f"Incorrect Predictions: {np.sum(y_pred != y_test.values)}")
print(f"Accuracy: {accuracy:.2%}")
print(f"\nDisease Distribution:")
print(f"No Disease: {np.sum(y_test == 0)} ({np.sum(y_test == 0)/len(y_test)*100:.
1f}%)")
print(f"Disease: {np.sum(y_test == 1)} ({np.sum(y_test == 1)/len(y_test)*100:.1
f}%)")
```

Out:

```
Total Test Samples: 308
Correct Predictions: 299
Incorrect Predictions: 9
Accuracy: 97.08%

Disease Distribution:
No Disease: 302 (98.1%)
Disease: 6 (1.9%)
```